Oracle C2M Log Analysis & Health Monitoring (SLO Assurance)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance) 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 5

Outcome . . . 6

1. Solution Overview — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance) 6

What C2M logging and health monitoring is . . . 6

Why this matters . . . 6

The C2M log landscape — three layers . . . 6

Health signals — what to watch . . . 7

Named objects this solution defines . . . 8

The SLO model . . . 8

What this is not . . . 8

The deployment model . . . 9

2. Requirements Guide — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance) 9

2.1 A reachable C2M environment and product user . . . 9

2.2 OUAF application log access (`$SPLEBASE/logs/system`) . . . 9

2.3 WebLogic server + domain log access (`$DOMAIN_HOME/servers/.../logs`) . . . 10

2.4 Batch / background-processing log access (`$SPLOUTPUT`) . . . 10

2.5 The health-signal set defined . . . 10

2.6 The SLO set defined and owned . . . 10

2.7 Time synchronization and ECID correlation . . . 11

2.8 Log retention, rotation, and disk headroom . . . 11

2.9 A non-production pilot environment and change control . . . 11

Checklist . . . 11

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 12

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 12

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 12

3. Environment topology (Dev / Test / Prod) & access . . . 12

4. Administrative access & tooling (WLST, shell, SQL) . . . 13

5. Change control, backup & a non-production pilot . . . 13

6. Collaborating teams & support model . . . 13

3. Implementation Guide — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance) 14

Step 1 — Assess the current log landscape . . . 14

Step 2 — Locate and standardize the logs (build the inventory) . . . 14

Step 3 — Define the health checks . . . 15
Step 4 — Define the early-warning signals . . . 15
Step 5 — Define the SLOs and error budgets . . . 16
Step 6 — Stand up the health report . . . 16
Step 7 — Scope the pilot . . . 16
Step 8 — Pilot the monitoring on non-production . . . 16
Step 9 — Validate against the pilot gate . . . 17
Step 10 — Expand to production . . . 17
5. Testing Guide — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance) 17
Test 1 — Every log layer is inventoried and readable . . . 17
Test 2 — Logs are current (collection is live) . . . 18
Test 3 — Health checks run and report . . . 18
Test 4 — Each early-warning signal fires on a known trigger . . . 18
Test 5 — Signals are clean when they should be . . . 18
Test 6 — Cross-layer correlation works . . . 18
Test 7 — The health/SLO report renders with error budgets . . . 18
Test 8 — Remediation preview is safe (dry-run) . . . 18
Pass/fail gate . . . 19
6. Troubleshooting Guide — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance) 19
A. A log layer cannot be found or read . . . 19
B. A log is stale (stopped being written) . . . 20
C. A signal never fires (false all-clear) . . . 20
D. A signal fires constantly (alert fatigue) . . . 20
E. Cross-layer correlation does not line up . . . 20
F. `OutOfMemoryError` or connection-pool exhaustion in the WebLogic log . . . 20
G. Log filesystem filling / logs growing too fast . . . 21
H. An SLO is breaching or its error budget is burning . . . 21
I. Diagnostics quick reference . . . 21
J. When to escalate . . . 21
7. Best Practices — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance) 21
Log hygiene . . . 21
Early detection . . . 22
SLO tracking . . . 22
Operating discipline . . . 23
8. Deliverables Summary — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance) 23
Documentation . . . 23

Named objects 24
Log landscape covered 24
Automation scripts (WLST/Jython, shell & SQL — not PowerShell) 24
What the buyer receives 25
Outcome 25
Appendix A — PowerShell Scripts 26

Executive Summary — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance)

Oracle Utilities Customer to Meter (C2M) writes its logs in three places — the OUAF application, the WebLogic Server middleware, and background **batch** — and the signal that keeps uptime high is buried across all three. When the logs are only read after a business user reports an outage, every problem is caught late: the connection pool was exhausted for an hour, the batch failed three nights running, the heap was climbing all week — each a missed chance to act before it became downtime. This solution establishes **C2M log analysis and health monitoring** the repeatable way: find and standardize the logs across all three layers, define the **health checks** and **early-warning signals** to watch, and track system health against **service-level objectives (SLOs)** — so you catch problems early and maximize uptime.

This is a complete, do-it-today package for an experienced C2M administrator. Every object is named once — the log inventory, the health-check set, the early-warning signal set, the SLO set, and the daily health report — and reused across environments and every monitoring cycle, so you configure it once and repeat it.

What you get

• A plain-language Solution Overview of the C2M log landscape — the OUAF application logs (`$SPLEBASE/logs/system/spl*.log`), the WebLogic server and domain logs (`$DOMAIN_HOME/servers/*/logs`), and the batch logs (`$SPLOUTPUT/threadpoolworker.<pool>.<datetime>-tech/-user.log`) — the health signals, and the SLO model, correlated across layers by ECID.

• A Requirements Guide with verify / implement-if-missing for every prerequisite — a reachable environment, read access to all three log layers, the named signal and SLO sets, time sync and ECID for correlation, and rotation/retention/disk headroom.

• An Implementation Guide of precise, ordered steps: locate and standardize the logs, define the health checks and early-warning signals, define the SLOs and error budgets, stand up the daily health report, then pilot, validate detection, and expand.

• Five automation scripts (shell, WLST/Jython, and SQL — not PowerShell) for read-only log/health discovery, health-check and signal validation, a health/SLO report, safe log rotation/cleanup remediation (dry-run by default), and a scope/target helper — authentic to C2M and WebLogic administration.

• Testing and Troubleshooting guides with explicit pass/fail gates (logs accessible, health tracked, SLOs measured), plus Best Practices for log hygiene, early detection, and SLO tracking.

What this delivers

C2M logs turned into provable health and uptime — every log inventoried and current, the early-warning signals watched, and health tracked against SLOs with error budgets.

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — proven on a non-production pilot before production, with a documented rollback for every production change.

Outcome

Reliable, observable C2M: log sprawl, late detection, and no SLO tracking replaced by an inventoried log landscape, watched early-warning signals, and health measured against SLOs — so problems are caught before they cost service, validated on a pilot before you scale.

1. Solution Overview — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance)

What C2M logging and health monitoring is

Oracle Utilities Customer to Meter (C2M) is a layered system — the Oracle Utilities Application Framework (OUAF) application, the Oracle WebLogic Server middleware that hosts it, and background **batch** processing — and each layer writes its own logs. The signal that keeps uptime high (a growing error rate, a filling connection pool, a batch that started failing, a heap heading for an `OutOfMemoryError`) is in those logs, but it is spread across three log landscapes and buried in volume. This solution establishes **C2M log analysis and health monitoring** the repeatable way: find and standardize the logs across all three layers, define a named set of **health checks** and **early-warning signals** to watch, and track system health against **service-level objectives (SLOs)** — so you catch problems early and maximize uptime, pilot-first.

> ***Version boundary.*** *Log paths, file names, environment variables, and object terminology below are from the OUAF Server Administration Guide, the C2M Server Administration Guide, and the WebLogic Server logging documentation. Version-specific details (C2M 25.4 / minimum v2.9, WebLogic 14c, the OUAF log format and directory layout) are* ***cited or flagged "verify for your version"*** *— never invented. Some notes and patches are gated on My Oracle Support (MOS) and are pointed to, not reproduced. See* `global-requirements-oracle.md`*.*

Why this matters

Logs that nobody reads until an outage are cost, not insurance. When the only time anyone looks at `spl*.log` or the WebLogic server log is after a business user reports the system is down, every problem is discovered late — the connection pool was exhausted for an hour, the batch failed three nights running, the heap was climbing all week — and each is a missed chance to intervene before it became downtime. A real solution turns the logs into leading indicators: it knows exactly where each log is, defines the specific signals that precede an outage (error-rate spikes, stuck-thread warnings, pool exhaustion, `OutOfMemoryError`, batch failures), watches them continuously, and measures the result against SLOs so "how healthy is C2M?" has a number, not an opinion. The fix is early detection, not faster firefighting.

The C2M log landscape — three layers

C2M writes logs in three places. Knowing each one — its path, its files, and what it tells you — is the foundation of everything else.

1. **OUAF application logs (**`$SPLEBASE/logs/system`**).** The framework's own system logs live under

the environment's SPLEBASE install directory in `logs/system` — the `spl*.log` files (and `spl_service.log`, which records SPL service times). These are independent of the WebLogic and database logs and carry the application-level errors, warnings, and service timings. (Paths per the OUAF Server Administration Guide — verify for your version.)

1. **WebLogic Server logs (`$DOMAIN_HOME/servers/<server>/logs`).** The middleware that hosts C2M

writes a **server log** per managed server (`<server-name>.log`) and an aggregated **domain log** in the Administration Server's log directory (`$DOMAIN_HOME/servers/<AdminServer>/logs/<domain>.log`). These carry JVM, deployment, JDBC/connection-pool, JMS, and stuck-thread messages — the health of the container C2M runs in. (Paths per the WebLogic Server logging documentation.)

1. **Batch / background-processing logs (`$SPLOUTPUT`).** The thread pool workers write batch logs to

the application output directory `$SPLOUTPUT`, split into a **technical** and a **business** log per pool run: `threadpoolworker.<threadpool>.<datetime>-tech.log` and `...-user.log`. These are where a failing or slow batch shows itself. (File-name pattern per the OUAF Server Administration Guide — verify for your version.)

The OUAF log line format is consistent across these — `<userid> <transactionid> <ECID> <date/time> <threadinfo> <log level> (<class>) <message>` — which is what makes them parseable: the **ECID** (Execution Context ID) is the correlation key that ties an application-layer entry to its WebLogic and batch entries for the same request or job.

Health signals — what to watch

From the three log landscapes this solution defines a named, reusable set of **health signals** — the things that precede an outage — so monitoring watches the same set everywhere:

- **Error/exception rate** — `ERROR`/`FATAL` lines per interval in the OUAF and WebLogic logs, and

any Java stack traces; a rising rate is the earliest general warning.

- `OutOfMemoryError` **/ heap pressure** — the single most important container signal; an OOM in the

server log means a JVM is (or is about to be) down.

- **Connection-pool health** — JDBC connection-pool exhaustion / "no resources currently available"

messages in the WebLogic log; a filling pool degrades the whole application before it fails.

- **Stuck threads** — WebLogic "stuck thread" warnings (a thread active past the configured max-stuck

time); repeated stuck-thread warnings precede a hung server.

- **Batch failures / overruns** — errored threads and long-running or non-completing runs in the

`threadpoolworker` logs; a batch that starts failing is often the first sign of a data or connectivity problem.

- **Log growth / disk** — the logs directories filling `$SPLOUTPUT` / the domain log volume; a full

log filesystem takes the application down on its own.

This solution names these once as the **health-signal set** and reuses it across environments so the same signals are watched in Dev, Test, and Production.

Named objects this solution defines

So the monitoring is repeatable, this solution names its artifacts once and reuses them:

- **Log inventory** — `C2M Log - Inventory - All Layers`, the documented map of every log (OUAF,

WebLogic, batch) with its path, owner, and retention.

- **Health-check set** — `C2M Health - Checks - Standard`, the standard set of health checks run

against the three layers.

- **Early-warning signal set** — `C2M Health - Signals - EarlyWarning`, the named signals above with

their thresholds.

- **SLO set** — `C2M SLO - Uptime - Standard`, the availability and health SLOs (see below) with their

targets and error budgets.

- **Health report** — `C2M Health - Report - Daily`, the standing daily health-and-SLO view.

The SLO model

Health is tracked against **service-level objectives** — explicit, measurable targets with an error budget — not against a vague sense of "seems fine":

- **Availability SLO** — the percentage of a measurement window the C2M application is up and serving

(for example, a target expressed as a monthly availability percentage). The **error budget** is the allowed downtime; watching it burn is itself an early warning.

- **Batch-completion SLO** — the percentage of scheduled batch cycles that complete successfully

within their window (a failed or overrunning nightly cycle burns this budget).

- **Error-rate SLO** — application/middleware error rate kept below a defined threshold over the

window.

- **Log-freshness / coverage SLO** — every in-scope log is being collected and is current (a log that

stopped being written is a blind spot, and a monitoring gap is itself a risk).

Each SLO has an owner, a target, a measurement source (which log and which signal), and a review cadence. Tracking them turns health from anecdote into a trend you can act on before the budget runs out. (Targets are yours to set from the business's uptime commitments — this solution defines the structure and the measurement, not the specific numbers.)

What this is not

This solution establishes log analysis and health monitoring against SLOs. It does not replace an enterprise APM/monitoring platform (where one such as Dynatrace exists, this complements it by defining what to watch and why), rewrite C2M business logic, or change the logging framework itself. Where the included scripts touch live systems they **read** — discovery, validation, and report are read-only; remediation (log rotation/cleanup preview) runs **dry-run by default**. Any change to production logging configuration, retention, or rotation goes through change control and is proven on a non-production environment first.

The deployment model

> *Assess the current log landscape and health blind spots, configure the log inventory plus the health checks, early-warning signals, and SLOs, pilot the monitoring on a non-production environment, validate that logs are accessible and signals fire against a gate, then expand to production. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand — with a documented rollback for every production change.*

2. Requirements Guide — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance)

Verify every requirement before building the log inventory, health checks, early-warning signals, or SLOs. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites (licensing, certified stack, environments, WLST/shell/SQL tooling, change control) are in `global-requirements-oracle.md`.

2.1 A reachable C2M environment and product user

Description: A licensed C2M environment with its OUAF product home reachable, and the OS product user that owns it (the account under which the online, batch, and service processes run), with the environment sourced via `splenviron.sh -e your-env`. Why: Every discovery and monitoring script needs the environment variables `splenviron.sh` sets — `SPLEBASE` (install/log home), `SPLOUTPUT` (batch/application output logs), and the paths to the OUAF utilities; without them the scripts cannot locate the logs. Verify: As the product user, run `splenviron.sh -e your-env`, then run `OC2MLOG-Discovery.sh` — it confirms the environment is sourced and `SPLEBASE`/`SPLOUTPUT` resolve. Implement-if-missing: Obtain the product-user credentials and the environment name; confirm the OUAF install is complete (`initialSetup`/`configureEnv` run) before proceeding.

2.2 OUAF application log access (`$SPLEBASE/logs/system`)

Description: Read access to the OUAF system logs under `$SPLEBASE/logs/system` — the `spl*.log` files (and `spl_service.log`) that carry application-level errors, warnings, and service timings. Why: The application-layer half of every health signal (error rate, exceptions, service timing) comes from these logs; without access there is no application health view. Verify: You can list and read `$SPLEBASE/logs/system` and see current `spl*.log` files; run `OC2MLOG-Discovery.sh`, which inventories them (read-only). Implement-if-missing: Grant the monitoring account read on the OUAF logs directory (or run as the product user); confirm the logs are current (being written) per the version's Server Administration Guide.

2.3 WebLogic server + domain log access (`$DOMAIN_HOME/servers/.../logs`)

Description: Read access to the WebLogic **server logs** (`$DOMAIN_HOME/servers/<server>/logs/<server>.log`) and the **domain log** in the Administration Server's log directory, plus (optionally) WLST access to read the logging configuration. Why: The middleware half of the health signals — JVM/`OutOfMemoryError`, JDBC connection-pool exhaustion, JMS, and stuck-thread warnings — lives in the WebLogic logs; these precede most container outages. Verify: You can read the managed-server logs and the domain log; run `OC2MLOG-Discovery.sh` (it locates them from `DOMAIN_HOME`) and, if WLST access is granted, `OC2MLOG-Validation.py` reads the logging config (read-only). Implement-if-missing: Grant read on `$DOMAIN_HOME/servers/*/logs` and, for config checks, a read-capable WebLogic user for WLST; confirm `DOMAIN_HOME` is known.

2.4 Batch / background-processing log access (`$SPLOUTPUT`)

Description: Read access to the thread pool worker batch logs in `$SPLOUTPUT` — the `threadpoolworker.<threadpool>.<datetime>-tech.log` and `...-user.log` files that record batch technical and business output. Why: Batch failures and overruns — often the first sign of a data, integration, or connectivity problem — show in these logs; a batch that starts failing is a leading indicator. Verify: You can list `$SPLOUTPUT` and read recent `threadpoolworker.*` logs; `OC2MLOG-Discovery.sh` inventories them (read-only). Implement-if-missing: Confirm `SPLOUTPUT` is set and readable (grant read or run as the product user); confirm batch logs are current for recent runs. File-name pattern per the OUAF Server Administration Guide — verify for your version.

2.5 The health-signal set defined

Description: The named early-warning signals this solution standardizes — `C2M Health - Signals - EarlyWarning`: error/exception rate, `OutOfMemoryError`/heap pressure, connection-pool exhaustion, stuck-thread warnings, batch failures/overruns, and log-growth/disk — each with a documented threshold and the log it is read from. Why: A named, consistent signal set is what lets monitoring watch the same things in every environment and turn raw log lines into early warning rather than after-the-fact forensics. Verify: Each signal has a source log and a threshold; run `OC2MLOG-Validation.sh`, which checks that the signal patterns are found (or cleanly absent) in the inventoried logs. Implement-if-missing: Define the signal set and thresholds (see the Implementation Guide) from the environment's own baseline; record them against `C2M Health - Signals - EarlyWarning`.

2.6 The SLO set defined and owned

Description: The service-level objectives this solution standardizes — `C2M SLO - Uptime - Standard`: availability, batch-completion, error-rate, and log-freshness/coverage — each with a target, an error budget, a measurement source (which log/signal), and an owner. Why: SLOs turn health from "seems fine" into a measured target with an error budget you can watch burn; without them there is nothing to track health *against*. Verify: Each SLO has a target, source, and owner; run `OC2MLOG-Report.sh`, which computes the current health/SLO view from the inventoried logs and signals. Implement-if-missing: Set the SLO targets from the business's uptime commitments and assign owners; record them against `C2M SLO - Uptime - Standard`.

2.7 Time synchronization and ECID correlation

Description: Synchronized clocks (NTP) across the OUAF, WebLogic, and database hosts, and the OUAF log format's **ECID** (Execution Context ID) available in the log lines, so entries across the three layers can be correlated for the same request or job. Why: Early detection often needs the *chain* — an application error, its WebLogic entry, and its batch entry; correlation by time and ECID only works if clocks agree and the ECID is present. Verify: Confirm NTP is in sync across the hosts and that log lines carry the ECID field (per the OUAF log format `<userid> <transactionid> <ECID> <date/time> ...`); `OC2MLOG-Validation.sh` samples a log line and reports whether the ECID field is populated. Implement-if-missing: Enable/verify NTP on all hosts; confirm the log format for your version includes the ECID.

2.8 Log retention, rotation, and disk headroom

Description: Known **rotation** (size/time), **retention** (how many files kept), and **disk headroom** for each log location — the OUAF `logs/system`, `$SPLOUTPUT`, and the WebLogic server/domain logs (WebLogic's `LogFileMBean` controls file size and count). Why: A log filesystem that fills takes the application down on its own, and logs that rotate away too fast destroy the evidence needed for early detection; both are health risks in their own right. Verify: You can state the rotation/retention and current disk usage per location; `OC2MLOG-Discovery.sh` reports log-directory sizes and `OC2MLOG-Validation.sh` flags low headroom. Implement-if-missing: Set WebLogic rotation via `LogFileMBean` (file min size, file count) and a retention/cleanup policy for `$SPLOUTPUT` and `logs/system`; use `OC2MLOG-Remediation.sh` (dry-run) to preview a safe cleanup/rotation.

2.9 A non-production pilot environment and change control

Description: A non-production C2M environment to build the inventory, signals, and SLOs and prove the monitoring before touching production, plus change control and backups (any logging-config change to WebLogic or OUAF) per the global requirements. Why: Logging-config, rotation, and retention changes are production-impacting; the model is Assess → Configure → Pilot → Validate → Expand with a documented rollback. Verify: A non-production environment is available and a backup/rollback path for any logging-config change is confirmed. Implement-if-missing: Stand up or obtain the non-production environment and establish the backup and change window before any production change.

Checklist

- C2M environment reachable, product user, `splenviron.sh -e your-env` sources cleanly — `OC2MLOG-Discovery.sh`
- OUAF application logs readable — `$SPLEBASE/logs/system` (`spl*.log`)
- WebLogic server + domain logs readable — `$DOMAIN_HOME/servers/*/logs`
- Batch logs readable — `$SPLOUTPUT` (`threadpoolworker.*-tech/-user.log`)
- Health-signal set defined — `C2M Health - Signals - EarlyWarning`, `OC2MLOG-Validation.sh`
- SLO set defined and owned — `C2M SLO - Uptime - Standard`, `OC2MLOG-Report.sh`

- Time sync (NTP) + ECID present for correlation
- Rotation/retention/disk headroom known per log location
- Non-production pilot + change control and backups

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

- **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.
- **Why required:** Installation media, patches, and support all flow from the entitlement.
- **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.
- **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

- **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).
- **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.
- **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).
- **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

- **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS

accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

- **Why required:** C2M is deployed and promoted across environments; each solution assumes the target

environment exists and you can reach WebLogic, the file system, and the database.

- **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration

Console / WLST, and the C2M database schema.

- **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

- **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic,

the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

- **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only

workflow; the solutions' automation is authored as WLST/shell/SQL.

- **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL

connectivity to the schema.

- **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

- **Description:** A **change-control** process for production changes, verified **backups** (domain

config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

- **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a

documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

- **How to verify:** Confirm a backup/restore path and a change window/approval for production.

- **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

- **Description:** Engagement with the **infrastructure, DBA, network, application-development, and

release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

- **Why required:** C2M administration is cross-functional; issues span middleware, database, network,

and application layers, and production support runs on a defined escalation and monitoring model.

- **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

3. Implementation Guide — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance)

Follow these steps in order. This guide establishes C2M log analysis and health monitoring: locate and standardize the logs across all three layers first, then define the health checks and early-warning signals, then track against SLOs, then validate — pilot-first on a non-production environment, then expand to production under change control. Object names are fixed — create them with the exact names shown and reuse them everywhere.

> *Model: assess the current log landscape and blind spots, configure the inventory, checks, signals, and SLOs, prove it on a non-production pilot, validate against the gate, then expand to production. Assess, then Configure, then Pilot, then Validate, then Expand — with a documented rollback for every production change.*
>
> *All commands assume the environment is sourced first:* `splenviron.sh -e your-env` *(run as the product user). Log paths, file names, and the OUAF log format are from the OUAF/C2M Server Administration Guides and the WebLogic Server logging documentation — verify the exact paths and file-name patterns for your version.*

Step 1 — Assess the current log landscape

1. Source the environment (`splenviron.sh -e your-env`) and run `OC2MLOG-Discovery.sh`. It captures:

the OUAF application logs under `$SPLEBASE/logs/system` (`spl*.log`), the batch logs in `$SPLOUTPUT` (`threadpoolworker.<pool>.<datetime>-tech.log` / `-user.log`), and — from `DOMAIN_HOME` — the WebLogic server and domain logs, with each location's file list and directory size (read-only).

1. Identify the blind spots: logs you cannot read, logs that stopped being written, a filesystem near

full, or a layer with no monitoring at all. Record gaps against the Requirements Guide checklist. Do not proceed past an unreadable log layer or a missing environment — implement those first.

Step 2 — Locate and standardize the logs (build the inventory)

1. Build the inventory `C2M Log - Inventory - All Layers` — one row per log with its path, the layer

(OUAF / WebLogic / batch), what it carries, its owner, and its rotation/retention:

• **OUAF application:** `$SPLEBASE/logs/system/spl*.log` (and `spl_service.log` for service times).

• **WebLogic server:** `$DOMAIN_HOME/servers/<server>/logs/<server>.log` per managed server.

• **WebLogic domain:** `$DOMAIN_HOME/servers/<AdminServer>/logs/<domain>.log` (aggregated).

• **Batch:** `$SPLOUTPUT/threadpoolworker.<threadpool>.<datetime>-tech.log` and `...-user.log`.

1. Confirm the log line format carries the **ECID** (`<userid> <transactionid> <ECID> <date/time>

<threadinfo> <log level> (<class>) <message>`) so entries can be correlated across layers, and that NTP keeps the hosts' clocks in step (Requirement 2.7). Standardize on the ECID as the correlation key.

Step 3 — Define the health checks

1. Create the health-check set `C2M Health - Checks - Standard` — the standing checks run against the

three layers:

• **Application health** — `spl*.log` current and error rate within threshold.

• **Container health** — WebLogic server logs free of `OutOfMemoryError`, connection-pool

exhaustion, and repeated stuck-thread warnings.

• **Batch health** — recent `threadpoolworker` logs show completed, not errored/overrunning, runs.

• **Collection health** — every inventoried log is current (being written) and the log filesystems

have headroom.

1. Each check names the log it reads and the pass condition. Run `OC2MLOG-Validation.sh` to exercise

the check set against the inventoried logs.

Step 4 — Define the early-warning signals

1. Create the early-warning signal set `C2M Health - Signals - EarlyWarning` — each signal is a

pattern in a named log with a threshold that means "act now, before it becomes downtime":

Signal	Read from	Fires on
Error/exception rate	OUAF `spl*.log`, WebLogic server log	`ERROR`/`FATAL` + stack traces per interval above baseline
`OutOfMemoryError` / heap	WebLogic server log	any `OutOfMemoryError` / repeated heap-pressure lines
Connection-pool exhaustion	WebLogic server log	"no resources currently available" / pool-at-max
Stuck threads	WebLogic server log	repeated "stuck thread" warnings
Batch failure / overrun	`$SPLOUTPUT/threadpoolworker.*`	errored threads or non-completing runs

Signal	Read from	Fires on
Log growth / disk	log filesystems	directory/volume above the headroom threshold

1. Set each threshold from the environment's own baseline (Step 1), not a guess — measure normal, then

set the trip point above it. `OC2MLOG-Validation.sh` reports whether each signal pattern is found (fired) or cleanly absent.

Step 5 — Define the SLOs and error budgets

1. Create the SLO set `C2M SLO - Uptime - Standard` — each SLO with a target, an error budget, a

measurement source, and an owner:

- **Availability** — % of the window C2M is up and serving (error budget = allowed downtime).
- **Batch-completion** — % of scheduled cycles completing successfully in-window.
- **Error-rate** — application/middleware error rate kept below the defined threshold.
- **Log-freshness / coverage** — every in-scope log current and collected (no blind spots).

1. Map each SLO to the signal(s) that measure it (availability ← OOM/pool/stuck signals + up/down;

batch-completion ← batch signal; error-rate ← error-rate signal; coverage ← collection-health check). Set targets from the business's uptime commitments and assign owners.

Step 6 — Stand up the health report

1. Build the daily health view `C2M Health - Report - Daily`: run `OC2MLOG-Report.sh` (read-only) to

render, from the inventoried logs and signals, the current health-check results, which early-warning signals fired, and each SLO's status and remaining error budget.

1. Make it the standing operational view so health and SLO burn are visible every day — a signal that

fired or a budget burning down is the early warning, caught in operations rather than by a business user.

Step 7 — Scope the pilot

1. Run `OC2MLOG-ScopeHelper.sh --scope pilot` to resolve the bounded set of log sources / hosts /

signals to bring under monitoring first (a representative slice: one managed server, the batch pool, the OUAF logs), then the full set for expansion. This keeps the pilot to a known scope rather than "everything at once".

Step 8 — Pilot the monitoring on non-production

1. On the non-production environment, run the full monitoring pass over the pilot scope: build/confirm

the inventory (`OC2MLOG-Discovery.sh`), exercise the checks and signals (`OC2MLOG-Validation.sh`), and render the health/SLO report (`OC2MLOG-Report.sh`).

1. Deliberately trigger a safe, known signal in non-production (for example, generate a handful of

test `ERROR` lines, or run a batch known to fail) and confirm the corresponding early-warning signal fires in the report — proving the monitoring *detects*, not just *collects*.

Step 9 — Validate against the pilot gate

1. Complete the Testing Guide. The pilot passes only when: every inventoried log across the three

layers is readable and current; each health check runs and reports; each early-warning signal fires on a known trigger and is clean otherwise; and the health report renders every SLO's status and error budget.

1. Remediate any gaps — an unreadable log, a filesystem short on headroom, an un-rotated log — with

`OC2MLOG-Remediation.sh` (dry-run by default: it previews the log rotation/cleanup or config change; add the apply flag only in a change window) before re-running validation.

Step 10 — Expand to production

1. After the pilot passes the gate, apply the same inventory (`C2M Log - Inventory - All Layers`),

checks (`C2M Health - Checks - Standard`), signals (`C2M Health - Signals - EarlyWarning`), and SLOs (`C2M SLO - Uptime - Standard`) to production under change control, with backups of any logging-config change taken first and a documented rollback ready.

1. Run `OC2MLOG-Report.sh` on the production cadence (daily) and keep `C2M Health - Report - Daily`

running so signals and SLO burn stay visible day over day — the loop that turns logs into maximized uptime.

Order of operations

```
Assess landscape -> Inventory logs -> Health checks -> Early-warning signals
-> SLOs & budgets -> Health report -> Scope pilot -> Pilot monitoring ->
Validate gate -> Expand to production
```

5. Testing Guide — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance)

Run after the pilot monitoring pass on the non-production environment (Implementation Step 8) and again after production expansion. Each test confirms one part of the log-analysis and health-monitoring chain works end to end — that it not only *collects* logs but *detects* problems and *measures* health against SLOs. Source the environment first: `splenviron.sh -e your-env`.

Test 1 — Every log layer is inventoried and readable

Do: Run `OC2MLOG-Discovery.sh`. Expected: The inventory lists OUAF application logs (`$SPLEBASE/logs/system/spl*.log`), WebLogic server and domain logs (from `DOMAIN_HOME`), and batch logs (`$SPLOUTPUT/threadpoolworker.*`) — each readable, with a current file present. No layer missing or unreadable (the `C2M Log - Inventory - All Layers` set).

Test 2 — Logs are current (collection is live)

Do: Check the newest file's timestamp per layer in the discovery output; run `OC2MLOG-Validation.sh`. Expected: Each in-scope log has been written recently (is current) — no log that stopped being written. A stale log is a blind spot and fails the collection-health check.

Test 3 — Health checks run and report

Do: Run `OC2MLOG-Validation.sh` (the `C2M Health - Checks - Standard` set). Expected: Application, container, batch, and collection health checks each run and report PASS/FAIL — `spl*.log` current and within error threshold, WebLogic logs free of OOM/pool-exhaustion/stuck-thread, recent batch runs completed, all logs current with disk headroom.

Test 4 — Each early-warning signal fires on a known trigger

Do: In non-production, trigger a safe, known instance of each signal (e.g. generate test `ERROR` lines; run a batch known to fail); re-run `OC2MLOG-Validation.sh` and `OC2MLOG-Report.sh`. Expected: The corresponding signal in `C2M Health - Signals - EarlyWarning` fires and appears in the report — error-rate, batch-failure (and, where safely reproducible, pool/stuck/OOM) detected. The monitoring detects, it does not merely store.

Test 5 — Signals are clean when they should be

Do: On a healthy run with no trigger, run `OC2MLOG-Validation.sh`. Expected: No signal fires falsely — a healthy environment reports all-clear. (A signal that fires constantly on a healthy system has its threshold set below baseline; re-tune per Implementation Step 4.)

Test 6 — Cross-layer correlation works

Do: Take a recent application error's ECID from `spl*.log` and find the matching WebLogic (and, if applicable, batch) entry by ECID/time. Expected: The ECID ties the entries across layers — the application error, its container entry, and any batch entry line up (Requirement 2.7). Correlation is possible, not blocked by clock skew or a missing ECID field.

Test 7 — The health/SLO report renders with error budgets

Do: Run `OC2MLOG-Report.sh` (the `C2M Health - Report - Daily` view). Expected: The report shows each health check's result, which signals fired, and each SLO's status and **remaining error budget** — availability, batch-completion, error-rate, and log-freshness/coverage. Health is a number and a trend, not an opinion.

Test 8 — Remediation preview is safe (dry-run)

Do: Run `OC2MLOG-Remediation.sh` with no apply flag against a log location needing rotation/cleanup. Expected: It prints the exact rotation/cleanup or config change it *would* make and changes nothing — safe to run anywhere. (Applying is a change-window action with `--apply`.)

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-8 pass on non-production: all layers inventoried, current, and readable; checks run; every signal fires on a known trigger and is clean otherwise; ECID correlation works; the report renders SLOs with error budgets; remediation dry-run is safe
Expand pass	Tests 1-3, 5, 7 pass in production after the first monitoring cadence, and Test 4 (signals fire) plus Test 8 (safe remediation) are verified in non-production before go-live

6. Troubleshooting Guide — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance)

Use `OC2MLOG-Discovery.sh` for the log inventory and currency, `OC2MLOG-Validation.sh` for the health checks and whether each early-warning signal fires, `OC2MLOG-Report.sh` for the health/SLO view, and `OC2MLOG-Remediation.sh` (dry-run by default) for safe log rotation/cleanup. Source the environment first: `splenviron.sh -e your-env`. MOS-gated notes are pointed to, not reproduced.

A. A log layer cannot be found or read

Symptom	Cause	Remediation
OUAF logs not found	`SPLEBASE` not set / wrong env	Re-source `splenviron.sh -e your-env`; confirm `$SPLEBASE/logs/system` exists and is readable; re-run `OC2MLOG-Discovery.sh`
Batch logs not found	`SPLOUTPUT` not set / no recent runs	Confirm `SPLOUTPUT` resolves and recent `threadpoolworker.*` files exist; if no runs, that is a batch gap, not a log gap
WebLogic logs not found	`DOMAIN_HOME` unknown / no read grant	Set/confirm `DOMAIN_HOME`; grant read on `$DOMAIN_HOME/servers/*/logs`; the domain log is under the Administration Server's log dir

B. A log is stale (stopped being written)

• A log's newest entry is old — collection is not live for that layer. Confirm the owning process is running (online server, thread pool worker, or Admin/managed server), that the process is writing to the expected path (custom log-file names can redirect it via user exits — verify for your version), and that the filesystem is not full (a full volume silently stops writes). A stale log is a blind spot and fails the collection-health check in `OC2MLOG-Validation.sh`.

C. A signal never fires (false all-clear)

Symptom	Cause	Remediation
Known error not detected	Threshold set too high, or wrong log	Re-baseline (Implementation Step 4) and lower the trip point; confirm the signal reads the correct log; re-test with a known trigger
Signal pattern never matches	Log format/level differs for your version	Verify the OUAF log format and the exact ERROR/FATAL wording for your version; adjust the pattern; re-run `OC2MLOG-Validation.sh`
Batch failure missed	Reading `-user.log` only, not `-tech.log`	Include both `threadpoolworker.*-tech.log` and `-user.log` — technical failures show in the tech log

D. A signal fires constantly (alert fatigue)

• A signal trips on every healthy run. Its threshold is below the environment's baseline. Measure normal first, then set the trip point above it (Implementation Step 4). A signal that always fires is as useless as one that never does — re-tune, do not silence it.

E. Cross-layer correlation does not line up

• An application error and its WebLogic entry will not correlate. Confirm **NTP** keeps the hosts' clocks in step (skew breaks time-based matching) and that the log lines carry the **ECID** field (`<userid> <transactionid> <ECID> <date/time> ...`). If the ECID is absent, verify the log format for your version; correlate on ECID first, time second. `OC2MLOG-Validation.sh` reports whether the ECID field is populated.

F. `OutOfMemoryError` or connection-pool exhaustion in the WebLogic log

• The container is at risk. An `OutOfMemoryError` means a JVM is (or is about to be) down; "no resources currently available" means the JDBC pool is exhausted and the application is degrading. These are top-priority signals: capture the server log around the event, engage the middleware/DBA teams on heap sizing or pool sizing, and treat as an incident. This solution *detects* early — the fix (heap/pool

tuning) is applied under change control, proven on non-production first.

G. Log filesystem filling / logs growing too fast

- A log directory is near full or growing faster than rotation. Confirm WebLogic rotation

(`LogFileMBean` file min size / file count) is set, and that `$SPLOUTPUT` and `logs/system` have a rotation/retention policy. Preview a safe rotation/cleanup with `OC2MLOG-Remediation.sh` (dry-run), then apply in a change window. Never delete or truncate an active log by hand — that is its own incident.

H. An SLO is breaching or its error budget is burning

- The health report shows an SLO at or past its budget. Read which signal is driving it (availability

← OOM/pool/stuck; batch-completion ← batch failures; error-rate ← error signal; coverage ← a stale log) and address the underlying signal. A budget burning faster than the window allows is the early warning — act on the burn rate before the SLO is missed.

I. Diagnostics quick reference

```
splenviron.sh -e your-env                          # source the environment first
./OC2MLOG-Discovery.sh                              # inventory OUAF/WebLogic/batch logs, sizes, currency (r
./OC2MLOG-Validation.sh                             # health checks + which early-warning signals fire (read
./OC2MLOG-Report.sh                                 # daily health/SLO view with error budgets (read-only)
./OC2MLOG-ScopeHelper.sh --scope pilot              # resolve the pilot log-source/host/signal scope
./OC2MLOG-Remediation.sh                            # dry-run log rotation/cleanup; add --apply in a change
```

J. When to escalate

If every log is inventoried and current, the signals are tuned to baseline, and a signal keeps firing (or an SLO keeps breaching) after the configuration is sound, the cause is below monitoring — a code, data, capacity, or infrastructure problem. Capture `OC2MLOG-Validation.sh`, `OC2MLOG-Report.sh`, and the relevant log excerpts (by ECID) and review with the middleware, DBA, and application teams. For product defects, consult the version's My Oracle Support notes and open a service request — gated content is pointed to, not reproduced here.

7. Best Practices — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance)

Log hygiene

- Inventory every log once and keep it current. Maintain `C2M Log - Inventory - All Layers` — OUAF

(`$SPLEBASE/logs/system`), WebLogic server/domain (`$DOMAIN_HOME/servers/*/logs`), and batch (`$SPLOUTPUT/threadpoolworker.*`) — with each log's path, owner, and retention. A log you did not know existed is a blind spot; an unknown log is an unmonitored one.

- Rotate before the filesystem fills, retain long enough to investigate. Set WebLogic rotation via

`LogFileMBean` (file min size and file count) and a size/time rotation plus retention policy for `$SPLOUTPUT` and `logs/system`. A full log filesystem takes the application down on its own; logs that rotate away too fast destroy the evidence you need for early detection — balance the two deliberately.

- Never let cleanup be ad-hoc. Use `OC2MLOG-Remediation.sh` (dry-run by default) to preview a rotation

or cleanup and apply it only in a change window — deleting the wrong log or truncating an active one is its own incident.

- Keep the ECID and NTP so logs correlate. Ensure the OUAF log format carries the **ECID** and that

NTP keeps the OUAF, WebLogic, and database hosts in step — early detection needs the *chain* across layers, and correlation breaks on clock skew or a missing ECID field.

- Standardize log paths and format across environments. Same locations and format in Dev, Test, and

Production so the same discovery, validation, and report scripts run everywhere unchanged.

Early detection

- Watch leading indicators, not just the outage. Define `C2M Health - Signals - EarlyWarning` on the

signals that *precede* downtime — error-rate rising, `OutOfMemoryError`, connection-pool exhaustion, stuck-thread warnings, batch failures — so you intervene before the business feels it.

- Set thresholds from a measured baseline. Measure normal first (Implementation Step 1), then set each

signal's trip point above it. A threshold guessed too low fires constantly and gets ignored; too high never fires — both are as bad as no monitoring.

- Treat `OutOfMemoryError` and pool exhaustion as top-priority signals. An OOM means a JVM is (or is

about to be) down; a filling connection pool degrades the whole application before it fails. These earn the fastest response.

- Correlate across layers before concluding. An application error, its WebLogic entry, and its batch

entry are one event — use the ECID to see the whole chain rather than fixing a symptom in one layer.

- Alert on the signal, review the trend. Fire an alert when a signal trips, but also review the

health report's trend — a slowly rising error rate or a steadily burning error budget is a warning even before any single threshold trips.

SLO tracking

- Track health against explicit SLOs with error budgets. Define `C2M SLO - Uptime - Standard`

(availability, batch-completion, error-rate, log-freshness/coverage), each with a target, a measurement source, and an owner — so "how healthy is C2M?" is a number and a trend, not an opinion.

- Watch the error budget burn, not only the breach. A budget burning faster than the window allows is

an early warning in itself; act on the burn rate before the SLO is missed.

• Make coverage an SLO too. A log that stopped being collected is a blind spot — track

log-freshness/coverage so a monitoring gap is caught like any other health problem.

• Own every SLO. Each SLO has a named owner and a review cadence; an SLO nobody owns is not tracked.

Operating discipline

• Keep the health report a standing daily view. Run `OC2MLOG-Report.sh` (`C2M Health - Report - Daily`)

every day so signals that fired and budgets burning down are seen in operations, not after an outage.

• Prove monitoring on a pilot before you scale. Confirm on non-production that signals fire on a known

trigger and the report renders SLOs (the pilot gate) before extending to production.

• Change logging config under change control only. Back up any WebLogic/OUAF logging-config change,

prove it on non-production first, and keep a documented rollback — rotation and retention changes are production-impacting.

• Complement, don't duplicate, an APM platform. Where an enterprise monitoring/APM tool exists, this

solution defines *what* to watch and *why* (the signals and SLOs) and feeds it — it does not replace it.

• Point to MOS for product defects. If a signal keeps firing after the config is sound, consult the

version's My Oracle Support notes and open a service request — gated content is pointed to, not reproduced.

8. Deliverables Summary — Oracle C2M Log Analysis & Health Monitoring (SLO Assurance)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The C2M log landscape (OUAF / WebLogic / batch), the health signals, and the SLO model (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Oracle)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: locate/standardize logs, define health checks and early-warning signals, track against SLOs, validate
5. Testing Guide	Validation procedures with pass/fail gates (logs accessible, health tracked, SLOs measured)

Document	Purpose
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations for log hygiene, early detection, and SLO tracking

Named objects

Object	Name
Log inventory (all layers)	`C2M Log - Inventory - All Layers`
Health-check set	`C2M Health - Checks - Standard`
Early-warning signal set	`C2M Health - Signals - EarlyWarning`
SLO set	`C2M SLO - Uptime - Standard`
Daily health/SLO report	`C2M Health - Report - Daily`

Log landscape covered

Layer	Location	Carries
OUAF application	`$SPLEBASE/logs/system/spl*.log` (+ `spl_service.log`)	Application errors, warnings, service timings
WebLogic server	`$DOMAIN_HOME/servers/<server>/logs/<server>.log`	JVM/OOM, JDBC pool, JMS, stuck threads
WebLogic domain	`$DOMAIN_HOME/servers/<AdminServer>/logs/<domain>.log`	Aggregated domain-level messages
Batch / background	`$SPLOUTPUT/threadpoolworker.<pool>.<datetime>-tech.log` / `-user.log`	Batch technical + business output

Automation scripts (WLST/Jython, shell & SQL — not PowerShell)

Script	What it does
`OC2MLOG-Discovery.sh`	Inventories the OUAF, WebLogic, and batch logs from `SPLEBASE`/`SPLOUTPUT`/`DOMAIN_HOME` with file lists, sizes, and currency; read-only health/log discovery
`OC2MLOG-Validation.sh`	Runs the standard health checks and reports whether each early-warning signal fires or is cleanly absent; read-only

Script	What it does
`OC2MLOG-Report.sh`	Renders the daily health/SLO view — check results, signals fired, and each SLO's status and error budget; read-only
`OC2MLOG-Remediation.sh`	Previews safe log rotation/cleanup (and rotation-config) to align a log location to policy; dry-run by default, applies only with an explicit flag in a change window
`OC2MLOG-ScopeHelper.sh`	Builds the in-scope log-source / host / signal target list for the pilot and each expansion wave

(An optional WLST helper `OC2MLOG-Validation.py` reads the WebLogic logging configuration read-only where WLST access is granted — referenced from the shell validation.)

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five automation scripts (shell / WLST-Jython / SQL) in a single ZIP.
- A repeatable monitoring practice following the model: Assess, then Configure, then Pilot, then Validate, then Expand.

Outcome

C2M logs turned into provable health and uptime — every log across the three layers inventoried and current, the early-warning signals watched, and health tracked against SLOs with error budgets — proven on a non-production pilot before it is expanded to production under change control.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MLOG-Discovery.sh

Purpose: Inventory the logs for an Oracle Utilities C2M environment across all three layers: the OUAF application logs (your-splebase/logs/system, spl*.log), the WebLogic server and domain logs (under your-domain-home/servers), and the batch logs (your-sploutput, the threadpoolworker per-pool tech and user logs). For each it reports the newest file, count, and directory size, and whether the log is current. Read-only — it changes nothing. Run it before defining checks or SLOs (Step 1). HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-Discovery.sh Optional: set DOMAIN_HOME=/path/to/your-domain to inventory the WebLogic logs. EXPECTED RESULTS Prints, per layer, the log directory, newest file + age, file count, and directory size. PASS = SPLEBASE/SPLOUTPUT resolve and each layer's log directory is present and readable with a current file. FAIL = environment not sourced or a log layer missing/ unreadable — fix before continuing. NOTES White-label, repeatable discovery — the same inventory every environment. Read-only where it reads live systems: it only lists and stats log files. Log paths and the threadpoolworker file-name pattern are from the OUAF/C2M Server Administration Guides and the WebLogic logging docs — verify exact paths/names for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-Discovery.sh Optional: set DOMAIN_HOME=/path/to/your-domain to inventory the WebLogic logs. EXPECTED RESULTS Prints, per layer, the log directory, newest file + age, file count, and directory size. PASS = SPLEBASE/SPLOUTPUT resolve and each layer's log directory is present and readable with a current file. FAIL = environment not sourced or a log layer missing/ unreadable — fix before continuing. NOTES White-label, repeatable discovery — the same inventory every environment. Read-only where it reads live systems: it only lists and stats log files. Log paths and the threadpoolworker file-name pattern are from the OUAF/C2M Server Administration Guides and the WebLogic logging docs — verify exact paths/names for your version.

OC2MLOG-Remediation.sh

Purpose: When a log location is filling or retaining more than policy, bring it back within policy by rotating/compressing old logs and removing files past the retention age — for the OUAF logs (your-splebase/logs/system), the batch logs (your-sploutput), and the WebLogic server/domain logs. DRY-RUN by default: it prints exactly which files it WOULD compress/remove and the WebLogic rotation setting to check, and changes nothing. It applies only when explicitly asked, and only intended for a change window. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-Remediation.sh # DRY-RUN: prints intended actions only ./OC2MLOG-Remediation.sh --apply # applies (change window only) Tune: RETAIN_DAYS=30 COMPRESS_DAYS=3 ./OC2MLOG-Remediation.sh EXPECTED RESULTS DRY-RUN lists the files older than RETAIN_DAYS it would remove and older than COMPRESS_DAYS it would gzip, per log location, plus the WebLogic LogFileMBean rotation settings to verify — and exits changing nothing. With --apply it compresses and removes those files (never an active/current log) and prints what it did. It never deletes the newest log and never truncates a file in use. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. It never rotates WebLogic itself (that is a LogFileMBean/console setting under change control) — it prints the setting to check. Only ever removes files past retention, never

the current log. Verify paths/retention for your version and policy before --apply.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-Remediation.sh # DRY-RUN: prints intended actions only ./OC2MLOG-Remediation.sh --apply # applies (change window only) Tune: RETAIN_DAYS=30 COMPRESS_DAYS=3 ./OC2MLOG-Remediation.sh EXPECTED RESULTS DRY-RUN lists the files older than RETAIN_DAYS it would remove and older than COMPRESS_DAYS it would gzip, per log location, plus the WebLogic LogFileMBean rotation settings to verify — and exits changing nothing. With --apply it compresses and removes those files (never an active/current log) and prints what it did. It never deletes the newest log and never truncates a file in use. NOTES White-label, repeatable remediation — DRY-RUN by default so it is safe to run anywhere. It never rotates WebLogic itself (that is a LogFileMBean/console setting under change control) — it prints the setting to check. Only ever removes files past retention, never the current log. Verify paths/retention for your version and policy before --apply.

OC2MLOG-Report.sh

Purpose: Render the standing health view (C2M Health - Report - Daily): the health-check results, which early-warning signals fired, and each SLO's status and remaining error budget (availability, batch-completion, error-rate, log-freshness/coverage). It reads an SLO definition file for targets and computes current status from the logs. Read-only — it changes nothing. Run it daily during the pilot and in production. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-Report.sh SLO targets file (default ./oc2mlog-slo.conf) lines: SLO,TARGET_PCT,SOURCE e.g. availability,99.5,uptime-signal Optional: DOMAIN_HOME=/path/to/your-domain to include WebLogic logs. EXPECTED RESULTS Prints three sections: (1) health-check results, (2) early-warning signals fired, (3) each SLO with target, current, status (MET/AT-RISK/BREACHED), and remaining error budget. PASS = every SLO MET and no signal fired. Any AT-RISK/BREACHED or fired signal is the early warning to act on. NOTES White-label, repeatable report — the same view every day, every environment. Read-only against live systems. Current-status figures are computed from the inventoried logs; the actual availability numerator (up/down) comes from your uptime source — the file lets you record it. Verify log paths/format for your version.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-Report.sh SLO targets file (default ./oc2mlog-slo.conf) lines: SLO,TARGET_PCT,SOURCE e.g. availability,99.5,uptime-signal Optional: DOMAIN_HOME=/path/to/your-domain to include WebLogic logs. EXPECTED RESULTS Prints three sections: (1) health-check results, (2) early-warning signals fired, (3) each SLO with target, current, status (MET/AT-RISK/BREACHED), and remaining error budget. PASS = every SLO MET and no signal fired. Any AT-RISK/BREACHED or fired signal is the early warning to act on. NOTES White-label, repeatable report — the same view every day, every environment. Read-only against live systems. Current-status figures are computed from the inventoried logs; the actual availability numerator (up/down) comes from your uptime source — the file lets you record it. Verify log paths/format for your version.

OC2MLOG-ScopeHelper.sh

Purpose: Produce a bounded, reusable scope for the monitoring work: which log sources (OUAF, WebLogic server/domain, batch), on which hosts, with which early-warning signals, to bring under monitoring first (pilot subset) and then in full (expansion). It reads a scope definition file and emits a clean, de-duplicated target list the other scripts and the monitoring cadence consume, so pilot and each expansion wave run against a known set rather than "everything at once". Read-only — it only reads the

scope file. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-ScopeHelper.sh --scope pilot # sources/hosts/signals tagged pilot ./OC2MLOG-ScopeHelper.sh --scope all # full expansion set Scope file (default ./oc2mlog-scope.conf) lines: LAYER,SOURCE,HOST,SIGNALS,SCOPE e.g. weblogic,server-log,host-a,oom+pool+stuck,pilot EXPECTED RESULTS Prints the resolved, de-duplicated target list for the requested scope: log sources with their layer and signals, and the distinct hosts. Use the output to target the discovery, validation, and report runs and to drive the monitoring cadence. If the scope file is missing it prints a template to create. NOTES White-label, repeatable scoping — the same file drives pilot then each expansion wave, so scope is explicit and auditable. Read-only. No live-system changes. Sources/hosts are yours to fill from OC2MLOG-Discovery.sh output; signals from C2M Health - Signals - EarlyWarning.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-ScopeHelper.sh --scope pilot # sources/hosts/signals tagged pilot ./OC2MLOG-ScopeHelper.sh --scope all # full expansion set Scope file (default ./oc2mlog-scope.conf) lines: LAYER,SOURCE,HOST,SIGNALS,SCOPE e.g. weblogic,server-log,host-a,oom+pool+stuck,pilot EXPECTED RESULTS Prints the resolved, de-duplicated target list for the requested scope: log sources with their layer and signals, and the distinct hosts. Use the output to target the discovery, validation, and report runs and to drive the monitoring cadence. If the scope file is missing it prints a template to create. NOTES White-label, repeatable scoping — the same file drives pilot then each expansion wave, so scope is explicit and auditable. Read-only. No live-system changes. Sources/hosts are yours to fill from OC2MLOG-Discovery.sh output; signals from C2M Health - Signals - EarlyWarning.

OC2MLOG-Validation.sh

Purpose: Run the standard health checks (C2M Health - Checks - Standard) and report whether each early-warning signal (C2M Health - Signals - EarlyWarning) fires in the inventoried logs: error/exception rate, OutOfMemoryError, connection-pool exhaustion, stuck threads, batch failures, and log currency. Read-only — it scans logs and reports PASS/FAIL/FIRED, it changes nothing. Run it on the pilot (Step 8) and after expansion. HOW TO RUN Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-Validation.sh Optional: DOMAIN_HOME=/path/to/your-domain to scan WebLogic logs; SCAN_MINS=1440 to bound how far back to scan (default 24h). EXPECTED RESULTS For each check prints PASS/FAIL, and for each signal FIRED or clear. Overall PASS = all logs current and readable and no signal FIRED. A FIRED signal is an early warning (act on it); a FAIL check is a monitoring gap to remediate (see OC2MLOG-Remediation.sh) before re-validating. NOTES White-label, repeatable validation — the same checks and signals every environment. Read-only where it reads live systems (greps and stats logs; it does not modify them). Signal patterns follow the OUAF log format and WebLogic message wording — verify the exact terms for your version and tune thresholds to your baseline.

How to run: Source the environment first, then run as the product user: splenviron.sh -e your-env ./OC2MLOG-Validation.sh Optional: DOMAIN_HOME=/path/to/your-domain to scan WebLogic logs; SCAN_MINS=1440 to bound how far back to scan (default 24h). EXPECTED RESULTS For each check prints PASS/FAIL, and for each signal FIRED or clear. Overall PASS = all logs current and readable and no signal FIRED. A FIRED signal is an early warning (act on it); a FAIL check is a monitoring gap to remediate (see OC2MLOG-Remediation.sh) before re-validating. NOTES White-label, repeatable validation — the same checks and signals every environment. Read-only where it reads live systems (greps and stats logs; it does not modify them). Signal patterns follow the OUAF log format and WebLogic message wording — verify the exact terms for your version and tune thresholds to your baseline.